

ayer 10:13 a.m.

 tadb_202620_examen02_performan...
PDF

 sabana_brechas_seguridad_lote_4.zip
Archivo

 sabana_brechas_seguridad_lote_3.zip
Archivo

 sabana_brechas_seguridad_lote_2.zip
Archivo

 sabana_brechas_seguridad_lote_1.zip
Archivo

bueno recuerda por favor el trabajo 1 lo que hicimos, y recuerda la bonificación, ya este trabajo me toca hacerlo a mí en la nube. ya mi amigo lo está haciendo en postgres y yo lo voy a hacer en mysql. recuerda que la vez pasada lo hicimos igual. él me lo pasa en postgres y yo lo hago en mysql pero tienen que quedar iguales. solo cambia el motor o como se le diga a eso



 Memoria actualizada

Sí, ya tengo claro el esquema de trabajo y lo voy a mantener para este Examen 02.

La dinámica es la misma del Examen 01: **Samuel trabaja en PostgreSQL y tú en MySQL**. El diseño y las consultas deben ser los mismos; lo que cambia es la implementación necesaria por las diferencias entre ambos motores. Eso coincide exactamente con el nuevo enunciado: si trabajan en pareja, cada estudiante usa un motor diferente, pero entregan el mismo diseño y las mismas consultas.

 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



Además, ya revisé el nuevo examen y hay un cambio importante para tu parte: esta vez quieres hacer **MySQL 8.x en la nube** para buscar la bonificación. El profesor permite infraestructura cloud siempre que no genere costos, y específicamente admite MySQL 8.x.

 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



Para la bonificación, el enunciado aclara que debemos demostrar que la infraestructura cloud fue **creada para esta actividad**, así que necesitaremos guardar evidencias desde el comienzo.

 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



También cambia el dominio: ahora trabajaremos con **brechas de seguridad**. Los cuatro ZIP contienen en total 40.000 filas y el modelo esperado termina en **cinco tablas**: organización, brecha, usuario, tipo de dato expuesto y una tabla puente que relaciona usuarios, brechas y tipos de dato.

 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



Y las dos consultas principales que finalmente deberán producir resultados equivalentes en PostgreSQL y MySQL son:

- **Etapas 3:** brechas de severidad alta/crítica del último año que expusieron al menos un dato de sensibilidad crítica, contando usuarios distintos afectados.

 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



- **Etapas 4:** dado un usuario, mostrar su historial de brechas, dato expuesto, sensibilidad, organización y fecha de notificación, ordenado cronológicamente.

 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



Y esta vez entra fuerte **performance tuning**: el script de implementación debe incluir índices y el script de consultas debe incorporar las mejoras de desempeño; además hay que entregar un Word con los planes de ejecución, donde se vean claramente árbol de operaciones, filas y costo.

 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



Regla que vamos a seguir: cuando Samuel te pase algo ↓ PostgreSQL, no vamos a rediseñarlo porque sí. Lo reproducimos funcionalmente en MySQL, adaptando sintaxis, tipos, índices, **EXPLAIN** u otras cosas únicamente

+ Mensaje para ChatGPT

Instantánea ▾



ahí te puse lo de inventario de bonificaciones que nosotros escogimos la de proyecto avanzado n1 que es la del examen 1 y 2 que ya hicimos el examen 2



Sí, y aquí hay una precisión importante. Revisé el **Inventario de proyectos adicionales** que habías subido y ya entiendo exactamente la bonificación que escogieron.

Ustedes eligieron el **Proyecto Avanzado No. 1: "Despliegue de base de datos en la Nube en exámenes No. 1 y No. 2"**, cuya bonificación es del 20 % sobre la nota definitiva.



tadb_202620_inventario_proyectos_adicionales_del_curso.pdf
PDF



La condición clave es esta: **solo uno de los dos estudiantes debe hacer la implementación cloud en cada examen y debe alternarse entre Examen 1 y Examen 2**, precisamente para que ambos adquieran la competencia.



tadb_202620_inventario_proyectos_adicionales_del_curso.pdf
PDF



Entonces ya entiendo lo que me estabas aclarando:

Examen 01: ya lo hicieron y tu compañero fue quien realizó la parte en la nube.

Examen 02: ahora te corresponde a ti realizar MySQL 8.x en la nube. No es simplemente que queramos usar cloud porque sí: es necesario para completar correctamente el **Proyecto Avanzado No. 1** que escogieron y cumplir la alternancia.

Además, para este Examen 02 el propio enunciado recalca que, si están optando por esa bonificación, hay que **demostrar infraestructura creada para esta actividad**.



tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



Por tanto, no debemos reutilizar simplemente la infraestructura anterior. Para tu parte vamos a **crear una infraestructura MySQL nueva en la nube para el Examen 02**, utilizando una capa gratuita sin costos y conectándola de forma segura desde un IDE como DBeaver, que son precisamente los requisitos del Proyecto Avanzado.



tadb_202620_inventario_proyectos_adicionales_del_curso.pdf
PDF



Y mantendremos la misma dinámica técnica del Examen 01:

Samuel → PostgreSQL

Sebastián → MySQL 8.x + nube en este examen

Mismo modelo → mismas consultas → mismos resultados esperados → implementación adaptada a cada DBMS.

Cuando Samuel nos vaya pasando su implementación PostgreSQL, la iremos llevando a MySQL sin cambiar el diseño lógico ni lo que hacen las consultas, salvo las adaptaciones inevitables de sintaxis y optimización propias de MySQL.

Así que el siguiente paso correcto es **crear tu MySQL en la nube**, tomando capturas desde el principio para que después tengamos la evidencia completa de la bonificación y del abastecimiento.

📄 📌 🔄 🗑️ 🔍 ...

no podemos ir montando como en la nube para que solo sea esperar que me pasa samuel

Sí, totalmente. De hecho, es lo mejor que podemos adelantar ahora.

No necesitamos esperar a Samuel para la infraestructura. Podemos dejar tu **MySQL 8.x en la nube completamente creado y conectado desde DBeaver**, y cuando Samuel te entregue modelo/scripts PostgreSQL, nos concentramos únicamente en adaptarlos a MySQL y cargar los cuatro lotes.

Además, esto encaja con la bonificación: tiene que ser **infraestructura cloud gratuita**, con conectividad segura desde el IDE, y en este Examen 02 debe evidenciarse que fue creada para esta actividad.



tadb_202620_inventario_proyectos_adicionales_del_curso.pdf
PDF



tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



Podemos adelantar hoy todo esto:

1. Crear la instancia MySQL 8.x en la nube.
2. Configurar usuario/credenciales y acceso de red.



Mensaje para ChatGPT

Instantánea ▾



 tadb_202620_inventario_proyectos_adicionales_del_curso.pdf
PDF



 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



Podemos adelantar hoy todo esto:

1. Crear la instancia MySQL 8.x en la nube.
2. Configurar usuario/credenciales y acceso de red.
3. Configurar la conexión segura.
4. Conectarla a DBeaver.
5. Probar la conexión con algo sencillo como `SELECT VERSION();`.
6. Crear inicialmente la base/esquema del examen si conviene, aunque yo esperaré a Samuel para las tablas, para garantizar que el modelo sea idéntico.
7. Ir tomando capturas de cada paso, que después utilizaremos en tu PDF de infraestructura.
8. Dejar preparado dónde cargaremos los cuatro ZIP/CSV de las 40.000 filas cuando tengamos el modelo definitivo. El examen confirma que son cuatro lotes y que el resultado esperado de la normalización son cinco tablas.

 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



¿Qué nube usamos?

Para esta bonificación están permitidos Microsoft, Amazon, Oracle y Google, siempre utilizando una capa gratuita que no genere costos.

 tadb_202620_inventario_proyectos_adicionales_del_curso.pdf
PDF



Como la vez pasada ya tuvieron experiencia con AWS pero ahora te corresponde a ti adquirir la competencia, podemos montar MySQL 8 en una opción cloud gratuita que cumpla literalmente el requisito.

Antes de empezar a crear cosas, si quiero verificar cuál de esos proveedores tiene ahora mismo una opción gratuita viable para MySQL 8 sin riesgo de cobros, porque esto puede cambiar. Después arrancamos juntos desde cero y vamos guardando las capturas que necesitarás para la evidencia.

porque no en aws?



Sí podemos hacerlo en AWS. De hecho, no hay ningún problema con repetir proveedor.

La bonificación no dice que ustedes tengan que alternar el proveedor cloud. Dice que pueden usar Microsoft, Amazon, Oracle o Google, y que debe ser una capa gratuita sin costos.

 tadb_202620_inventario_proyectos_adicionales_del_curso.pdf
PDF



Lo que sí debe alternarse es quién hace la implementación en la nube: uno de ustedes en el Examen 1 y el otro en el Examen 2.

 tadb_202620_inventario_proyectos_adicionales_del_curso.pdf
PDF



Entonces perfectamente podemos hacer:

Examen 01

- Samuel → PostgreSQL en AWS ☁

Examen 02

- Sebastián → MySQL 8.x en AWS ☁

Y así cumplen exactamente el Proyecto Avanzado No. 1.

Además, para este examen tenemos que crear una infraestructura nueva, porque el enunciado dice expresamente que para optar por la bonificación debe demostrarse infraestructura creada para esta actividad.

 tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



Así que AWS me parece una muy buena opción, especialmente porque ya conocemos el flujo general del examen anterior.

Podemos empezar ya con AWS → MySQL 8.x → DBeaver, tomando capturas desde la creación para que después el PDF de evidencia prácticamente se arme solo.

Perfecto. Vamos con **AWS RDS + MySQL 8.4**. AWS confirma actualmente soporte para MySQL 8.4 y 8.0.

 Amazon Web Ser...

Antes de crearla, ojo con el costo: AWS cambió su Free Tier en 2025. Dependiendo de cuándo se creó tu cuenta, puedes ver **Free tier** o **Sandbox/créditos**; por eso vamos a mirar muy bien lo que indique tu consola y no activar nada que genere cobros.  Documentación d... +1

Empecemos

Entra a la consola de AWS y busca:

RDS

Entra a **Amazon RDS → Databases → Create database**. AWS documenta exactamente ese flujo para crear una instancia MySQL.  Documentación d...

[Abrir Amazon RDS](#) 

Cuando llegues a **Create database**, todavía **no le des a crear**.

Selecciona inicialmente:

 Texto sin formato

Database creation method:
Standard create

Engine type:
MySQL

Usaremos **Standard create** porque necesitamos controlar configuración, seguridad y costos, además de documentarlo bien para el examen.

Primera evidencia

Desde este momento empieza a tomar capturas. Guarda una donde se vea:

Amazon RDS → Create database → Standard create → MySQL

Esto nos servirá después para demostrar que la infraestructura fue creada específicamente para este Examen 02, que es un requisito explícito para la bonificación.



tadb_202620_examen02_performance_tuning_20260921.pdf
PDF



No avances todavía con las demás opciones. Mándame una captura de cómo te aparece **Create database**, porque AWS muestra opciones diferentes dependiendo del tipo y antigüedad de la cuenta, y ahí escogemos exactamente las que no nos generen costos.

     ...



Perfecto, estás exactamente donde necesitamos. Además, “Bases de datos (0)” nos sirve como buena evidencia de que estamos creando la infraestructura desde cero para este examen.

Ahora haz clic en el botón naranja “Crear base de datos”.

En la siguiente pantalla deberían aparecer opciones como:

- Método de creación
- Motor
- Versión del motor
- Plantilla
- Disponibilidad
- Configuración de la instancia
- Almacenamiento
- Conectividad

No confirmes la creación todavía. Mándame una captura de la parte superior de esa pantalla y vamos configurándola juntos, opción por opción, empezando por **MySQL 8.x** y verificando qué opciones de capa gratuita te ofrece tu cuenta.



Ahí escoge **Configuración completa**.

No uses **Configuración exprés**, porque para el examen necesitamos controlar bien:

- MySQL 8.x.
- Tipo/tamaño de instancia.
- Almacenamiento.
- Acceso desde DBeaver.
- Configuración de red y seguridad.
- Y, sobre todo, comprobar qué opciones entran en la capa gratuita antes de crearla.

Entonces:

Crear base de datos → Configuración completa

Haz clic ahí y mándame la siguiente pantalla. No crees todavía la instancia.



Perfecto. Esta pantalla confirma algo muy bueno: arriba AWS te está indicando explícitamente que tu cuenta está en “**plan gratuito**”. Guarda esta captura como evidencia.

Ahora selecciona **MySQL** —el cuadro con el delfín—, **no Aurora MySQL**, porque el examen te asigna específicamente MySQL 8.x.

Después de seleccionarlo, baja un poco. Te deberían aparecer **versión del motor** y otras opciones.

Por ahora haz solamente esto:

Tipo de motor → MySQL

Luego mándame captura de lo que aparece inmediatamente debajo y escogemos la versión correcta antes de seguir.



Bien, aquí ya vemos claramente “**Capa gratuita**”, **db.t4g.micro**, 2 vCPU, 1 GiB RAM y 20 GiB. Esa es la opción que queremos mantener. **No cambies esa selección**.

Pero cambia una cosa antes de continuar: arriba tienes marcada “**Creación sencilla**”. Para el examen prefiero “**Configuración completa**”, porque necesitamos controlar y evidenciar la versión exacta de MySQL, conectividad, seguridad y demás parámetros.

Haz clic en **Configuración completa**. Es posible que ci ↓ e bastante la pantalla.

Después verifica que sigas siendo:

+ Mensaje para ChatGPT

Instantánea



Configuración de la instancia

Las opciones de configuración de la instancia de base de datos que aparecen a continuación están limitadas a las que admite el motor que ha seleccionado anteriormente.

Clase de instancia de base de datosInformación

Ocultar filtros

Mostrar las clases de instancia que admiten las escrituras optimizadas de Amazon RDS

Información

Las escrituras optimizadas de Amazon RDS mejoran el rendimiento de escritura hasta 2 veces sin costo adicional.

Incluir clases de generación anterior

...

Mostrar más ▾



Perfecto. Déjalo exactamente en `db.t4g.micro`.

Eso coincide con la opción de **capa gratuita** que AWS te mostró antes:

- `db.t4g.micro`
- 2 vCPU
- 1 GiB RAM
- Arquitectura ARM/Graviton
- Más que suficiente para este examen y las 40.000 filas.

No actives filtros de clases estándar, memoria ni generaciones anteriores. Tampoco necesitamos Multi-AZ.

📸 Toma captura de esta sección, porque sirve para demostrar que elegiste una instancia pequeña correspondiente al plan gratuito.

Ahora baja a **Almacenamiento**. Ahí hay que revisar con cuidado:

- tipo de almacenamiento,
- cantidad de GiB,
- IOPS,
- escalado automático.

Mándame lo que te aparece en **Almacenamiento** antes de cambiarlo.



Almacenamiento

Tipo de almacenamiento

Información

Los volúmenes de almacenamiento SSD de IOPS aprovisionadas (io2) ya están disponibles.

SSD de uso general (gp2) Rendimiento de referencia determinado por el tamaño del volumen

Almacenamiento asignado

Información

GiB

El valor de almacenamiento asignado debe ser de 20 GiB a 6,144 GiB

...

Mostrar más ▾

Aquí lo dejamos lo más controlado posible:

- Tipo de almacenamiento: SSD de uso general (gp2) ✓
- Almacenamiento asignado: 20 GiB ✓, que es el mínimo que te permite.
- Habilitar escalado automático de almacenamiento: NO ✗.

Ese último es importante: la propia pantalla te advierte que se aplican cargos cuando escala automáticamente, y para nuestra bonificación queremos mantenernos dentro de infraestructura sin costos adicionales.

Entonces debería quedar:

</> Texto sin formato



SSD de uso general (gp2)

Almacenamiento asignado: 20 GiB



+ | Mensaje para ChatGPT

Instantánea ▾





Si, así está perfecto. La captura confirma exactamente lo que queríamos:

- SSD de uso general (gp2) ✓
- 20 GiB ✓
- Escalado automático desactivado ✓

📸 Guarda esta captura como evidencia, está muy buena porque se ven las tres configuraciones claramente.

No cambies nada aquí. Baja a la siguiente sección. Cuando llegues a **Conectividad o Disponibilidad y durabilidad**, mándame captura/texto antes de configurar. Esa parte será importante porque necesitamos que después DBeaver desde tu Mac pueda conectarse al MySQL de AWS.

📄 📌 ↺ 🔍 ⋮

Conectividad Información

Recurso de computación

Seleccione si desea configurar una conexión a un recurso de computación para esta base de datos. Al establecer una conexión, se cambiará automáticamente la configuración de conectividad para que el recurso de computación se pueda conectar a esta base de datos.

No se conecte a un recurso informático EC2

No configure una conexión a un recurso informático para esta base de datos. Puede configurar manualmente una conexión a un recurso informático más adelante.

Conectarse a un recurso informático de EC2

Configure una conexión a un recurso informático EC2 para esta base de datos.

Nube privada virtual (VPC)

Información

Elija la VPC. La VPC define el entorno de red virtual para esta instancia de DB.

...

Mostrar más ▾



Perfecto. Aquí configuramos pensando en dos cosas: **conectarte desde DBeaver en tu Mac y no activar servicios adicionales con costo**.

Déjalo así:

- **Recurso de computación:** No se conecte a un recurso informático EC2 ✓
No necesitamos EC2; DBeaver se conectará directamente a RDS.
- **VPC:** Default VPC ✓
- **Grupo de subredes:** predeterminado ✓
- **Acceso público:** Sí ✓
Esto es necesario para conectarnos desde tu Mac. Después restringiremos el firewall para que no quede MySQL abierto a todo Internet.
- **Grupo de seguridad VPC:** selecciona Crear nuevo. Ponle, por ejemplo: `examen02-mysql-sg`

Luego verificaremos que el puerto 3306 permita únicamente tu IP pública, no 0.0.0.0/0.
- **Zona de disponibilidad:** Sin preferencia ✓
- **Proxy de RDS:** NO crear proxy ✗. La propia consola advierte que tiene costos adicionales.
- **Entidad de certificación:** deja `rds-ca-rsa2048-g1` ✓. Nos interesa especialmente porque el proyecto adicional exige conectividad segura/cifrada desde el IDE.

En resumen:

</> Texto sin formato



```
EC2: No
VPC: Default VPC
Subred: predeterminado
Acceso público: Sí
Security Group: Crear nuevo
Nombre SG: examen02-mysql-sg
Zona disponibilidad: Sin preferencia
RDS Proxy: No
CA: rds-ca-rsa2048-g1
```

📸 Esta sección es una evidencia importante, sobre todo donde se vea el acceso público, grupo de seguridad y certificado.

Un detalle: "Acceso público: Sí" no significa que vamos a dejar la base abierta para cualquiera. El acceso real lo controlaremos con el Security Group y limitaremos el 3306 a tu IP.

Configura eso y sigue bajando. Todavía no pulses "Crear base de datos". Mándame la siguiente sección que te aparezca.

+ Mensaje para ChatGPT

Instantánea ▾ 🔍 ↗

Supervisión

Información

Elija la configuración de monitorización para esta base de datos. El modo estándar de Database Insights ofrece siete días de retención para obtener métricas detalladas de la base de datos sin costo adicional. El modo avanzado ofrece características de supervisión a nivel de flota y de primera calidad, incluida la retención a largo plazo de las métricas de la base de datos. Los precios de Database Insights son independientes de las estimaciones mensuales de RDS. Consulte [Precios de Amazon CloudWatch](#).

Database Insights: avanzado

- Monitorización a nivel de flota
- Integración con CloudWatch Application Signals
- Métricas detalladas por base de datos y consulta (retención prolongada sin costo adicional)

...

Mostrar más ▾

Aquí vamos a mantenerlo sencillo y evitar extras.

Configúralo así:

- Database Insights: Estándar . La propia consola indica que tiene 7 días de métricas detalladas sin costo adicional. No necesitamos el avanzado.
- Monitorización mejorada: desactivada . No la necesitamos para el examen.
- Exportaciones de registros a CloudWatch: deja todas sin marcar por ahora. Para el análisis de performance utilizaremos principalmente las herramientas de MySQL y los planes de ejecución que exige el examen.
- Rol IAM: no necesitamos configurar nada adicional si no habilitamos esas exportaciones.

Debe quedar básicamente:

✎ Texto sin formato

```
Database Insights:      Estándar
Monitorización mejorada: No
Auditoría:              No
Errores:                 No
General:                 No
IAM DB auth error:      No
Consultas lentas:        No
```

Con Database Insights estándar ya tenemos monitorización básica suficiente en AWS sin añadir complejidad.

Ahora baja a la siguiente sección, que probablemente será Configuración adicional. Esa sí me interesa verla completa antes de crear la instancia, porque ahí suelen aparecer copias de seguridad, mantenimiento, cifrado, eliminación y otras opciones.

📄 ↶ ↷ 🔍 ⋮

listo terminamos creo la base?

📄 ✎

Sí, si ya terminaste las secciones y mantuviste lo que configuramos, ya podemos crearla.

Antes de pulsar Crear base de datos, haz una última revisión rápida:

- MySQL Community 8.4.9
- Instancia db.t4g.micro
- 20 GiB gp2
- Escalado automático desactivado
- Multi-AZ no
- Acceso público Sí
- Security Group nuevo examen02-mysql-sg
- RDS Proxy no
- Certificado rds-ca-rsa2048-g1
- Database Insights Estándar
- Monitorización mejorada desactivada
- Soporte extendido de RDS desactivado
- Que abajo no aparezca ningún aviso de que estás activando alguna característica de pago adicional.

Si todo está así, sí: dale a "Crear base de datos". 

📸 Y guarda una captura justo antes de crearla si puedes. Después AWS tardará unos minutos y debería aparecer con estado Creando. Cuando te aparezca la instancia examen02-mysql, mándame captura y seguimos con seguridad → endpoint → conexión desde DBeaver.

📄 ↶ ↷ 🔍 ⋮



+ Mensaje para ChatGPT

Instantánea ▾





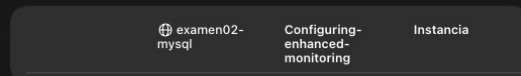
Perfecto, ya quedó creándose correctamente. ✅ En la captura se ve:

- Instancia: `examen02-mysql`
- Estado: **Creando**
- Motor: **MySQL Community**
- Tamaño: `db.t4g.micro`

Guarda esta captura como evidencia del abastecimiento en AWS.

Ahora no tenemos que tocar nada mientras diga “Creando”. Espera unos minutos y actualiza con ↻ hasta que el estado cambie a **Disponible**.

Cuando diga **Disponible**, haz clic sobre `examen02-mysql`. Ahí vamos a sacar el **endpoint** y **puerto 3306**, revisar el **Security Group** y finalmente conectarlo a **DBeaver**, igual que hicimos en el trabajo anterior pero esta vez contra MySQL en RDS.



Va bien. Ese estado **Configuring-enhanced-monitoring** significa que AWS todavía está terminando de **configurar la instancia**; todavía no intentemos conectarnos desde DBeaver.

Veo además:

- `examen02-mysql` ✅
- **MySQL Community** ✅
- `us-east-2a` (Ohio) ✅
- `db.t4g.micro` ✅
- VPC correcta ✅
- CPU ~38 % durante el aprovisionamiento: normal.
- 0 conexiones: normal porque aún no hemos conectado DBeaver.

Eso sí: aparece **Configuring-enhanced-monitoring**, aunque habíamos pensado dejar la monitorización mejorada desactivada. Cuando termine la creación revisamos si efectivamente quedó habilitada; no hace falta modificar nada mientras está configurándose.

Espera hasta que Estado diga **Disponible / Available**. En cuanto aparezca, entra haciendo clic en `examen02-mysql` y me muestras la sección **Conectividad y seguridad**. Ahí hacemos el paso importante: **endpoint + puerto + Security Group** → DBeaver.



ya sale dispo



Perfecto. Entonces la instancia **ya está lista**. Ahora vamos con la conexión desde DBeaver, pero primero necesitamos los datos de RDS.

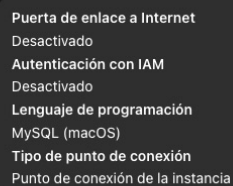
Haz clic en `examen02-mysql` y entra a la pestaña/sección “**Conectividad y seguridad**”.

Ahí busca y dime qué aparece en:

- **Punto de enlace / Endpoint**
- **Puerto** — debería ser `3306`
- **Accesible públicamente** — debería decir **Sí**
- **Grupos de seguridad de VPC** — debería aparecer el que creamos.

⚠ El endpoint sí me lo puedes mostrar; no es una contraseña. No me envíes la contraseña maestra.

Si puedes, mándame una captura de **Conectividad y seguridad**. Antes de abrir DBeaver quiero comprobar el **Security Group**, porque necesitamos permitir `3306` **solo desde tu IP**, no desde todo Internet.



Ah, ahí encontramos algo importante: “**Puerta de enlace a Internet: Desactivado**”. Eso explica que todavía no podamos plantear la conexión directa desde DBeaver e. ↴ Mac.



Perfecto. Ya quedó conectada correctamente. ✅

La captura además es una evidencia excelente porque muestra:

- Connected (3441 ms).
- Servidor: MySQL 8.4.9.
- Driver: MySQL Connector/J 8.2.0.
- Y venimos de configurar la conexión con el certificado CA de AWS y SSL.

Guarda esta captura sí o sí para la documentación de abastecimiento/conectividad.

Ahora dale OK → Finish para guardar la conexión en DBeaver.

Con esto ya tenemos lista la infraestructura base:

AWS RDS → MySQL 8.4.9 → Security Group restringido → SSL/TLS → DBeaver. ✅

El siguiente paso puede ser preparar la base para que, cuando Samuel te pase su implementación de PostgreSQL, podamos adaptar la misma solución a MySQL manteniendo modelo, datos y resultados equivalentes, como hicimos en el Examen 01.

📄 📄 📄 📄 ...

 **tadb202620_examen_02-main.zip**
Archivo

 **Texto pegado.txt**
Documento

```
mira el zip, la implementacion esta en postgres/sql.
\-- Examen 02 - Topicos Avanzados de Base de Datos
\-- Etapa 3 - Brechas críticas del ultimo ano
\-- Samuel Parra Cadavid
\-- ID SIGAA: 000551191
\-- PostgreSQL 18.x

\-- Consulta base
select
&#x20; o.nombre_organizacion,
&#x20; b.codigo_brecha,
&#x20; b.fecha_deteccion,
&#x20; b.vector_ataque,
&#x20; count(distinct e.codigo_usuario) as usuarios_afectados
from brechas_seguridad.brecha b
join brechas_seguridad.organizacion o
&#x20; on o.id_organizacion = b.id_organizacion
join brechas_seguridad.exposicion_usuario_brecha e
&#x20; on e.codigo_brecha = b.codigo_brecha
...
```

Mostrar más ▾

📄 📄

Sí. Ya ubiqué la implementación completa de Samuel en PostgreSQL y confirma la estructura que necesitamos replicar en MySQL. El script crea el esquema `brechas_seguridad`, las 5 tablas normalizadas y la tabla staging; después carga y normaliza los datos.

 **Texto pegado.txt**
TXT 

 **Texto pegado.txt**
TXT 

Además, la implementación de Samuel establece estos resultados esperados después de importar los cuatro CSV:

- `organizacion` : 60
- `tipo_dato_expuesto` : 10
- `brecha` : 800
- `usuario_afectado` : 7.887
- `exposicion_usuario_brecha` : 40.000

 **Texto pegado.txt**
TXT 

Y sí: lo que me pegaste corresponde a los otros dos scripts, **Etapa 3** y **Etapa 4**.

Por tanto, nuestra carpeta debería terminar así:



+ | Mensaje para ChatGPT Instantánea ▾ 🗣️ ⬆️

vamos a hacerlo paso a paso para tener los 3



Sí. Vamos uno por uno, sin adelantarnos, y verificamos cada uno en DBeaver antes de pasar al siguiente.

El orden será:

1. **01_implementacion.sql** → convertir la implementación de Samuel de PostgreSQL a MySQL 8.4.9, ejecutarla, importar los CSV y validar que los conteos den exactamente 60, 10, 800, 7887 y 40000. Esos son los valores esperados del original.

Texto pegado.txt
TXT



2. **Etapa 3** → pasar la consulta, los dos índices y los planes de ejecución a MySQL, comprobando que la consulta funcional dé exactamente los mismos registros que PostgreSQL.
3. **Etapa 4** → hacer lo mismo con el historial del usuario `US-002823`.

Así al final tendremos los 3 **.sql** de MySQL y sus respectivas evidencias/resultados.

Empecemos por el 1: implementación

El PostgreSQL de Samuel empieza creando usuario/rol y esquema, y después crea las tablas.

Texto pegado.txt
TXT



En nuestro caso no vamos a copiar eso literalmente porque estamos en AWS RDS MySQL y ya entramos como `admin`.

Primero quiero que abras tu conexión `examen02-mysql` en DBeaver → SQL Editor → New SQL Script.

Ahí ejecuta **solamente esto**:

```
</> SQL
CREATE DATABASE brechas_seguridad
CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci;

USE brechas_seguridad;
```



Luego ejecuta:

```
</> SQL
SELECT DATABASE();
```



Debe aparecer:

```
</> Texto sin formato
brechas_seguridad
```



No hagamos todavía las tablas. Ejecuta eso y mándame qué te sale; de ahí seguimos con el siguiente bloque del `01_implementacion.sql`.



Sí, estás exactamente donde necesitamos: un SQL Script nuevo conectado al RDS `examen02-mysql`. ✓

Pega este primer bloque:

```
</> SQL
CREATE DATABASE brechas_seguridad
CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci;

USE brechas_seguridad;

SELECT DATABASE();
```



Luego ejecútalo con el botón  de arriba a la izquierda. Si DBeaver te pregunta qué ejecutar, selecciona **Execute SQL Script**.

Al final, en el resultado de:



```
</> SQL
+ | Mensaje para ChatGPT
```

Instantánea



Texto pegado.txt
Documento

mira como lo tiene el

Sí. Ya viendo el archivo completo, ahora queda claro cómo lo hizo Samuel, y hay una corrección importante a lo que te dije antes.

Samuel no tiene un script que se ejecuta entero de una sola vez desde cero hasta el final. Él mismo especifica que está organizado por bloques y que hay una pausa para importar los cuatro CSV desde DBeaver.

Texto pegado.txt
TXT

La estructura exacta es:

1. Bloque 1 — Usuario, esquema y privilegios. Crea el rol `samuel`, le da acceso a `examen02`, crea el esquema `brechas_seguridad` y configura el `search_path`.

Texto pegado.txt
TXT

2. Bloque 2 — Tablas normalizadas. Crea `organizacion`, `brecha`, `usuario_afectado`, `tipo_dato_expuesto` y `exposicion_usuario_brecha`, con PK, FK, `UNIQUE` y `CHECK`.

Texto pegado.txt
TXT

3. Bloque 3 — Staging. Crea `staging_brechas` con 19 columnas en texto.

Texto pegado.txt
TXT

4. PAUSA: importa manualmente los 4 CSV desde DBeaver en modo `Append/Insert`. El resultado esperado son 40.000 filas.

Texto pegado.txt
TXT

5. Bloque 4 — Carga normalizada. Toma los datos de staging y llena las cinco tablas definitivas haciendo las conversiones necesarias.

Texto pegado.txt
TXT

6. Bloque 5 — Validaciones. Comprueba conteos, duplicados, nulos, claves foráneas huérfanas y calidad de datos.

Texto pegado.txt
TXT

Y los conteos finales que él espera son:

</> Texto sin formato

```
organizacion          60
tipo_dato_expuesto    10
brecha                 800
usuario_afectado      7887
exposicion_usuario_brecha 40000
```

Texto pegado.txt
TXT

Entonces nosotros debemos replicarlo en MySQL

No debemos inventarnos otra metodología. Hacemos la misma implementación, pero adaptando lo que PostgreSQL no soporta igual en MySQL.

Y como ya tenemos creada y seleccionada `brechas_seguridad` en RDS, vamos paso a paso igual que Samuel.

Nuestro siguiente paso es el Bloque 1 adaptado a MySQL. Después hacemos las 5 tablas, luego staging, importamos los 4 CSV y seguimos.

Así al final tendremos nuestro `01_implementacion.sql` organizado prácticamente igual al de Samuel, pero funcionando en MySQL 8.4.9 sobre AWS RDS.

📄 ⬆️ ↺ 📄 🔍 ...

vamos damelo



Sí. Arranquemos con nuestro Bloque 1 en MySQL, equivalente al primer bloque de Samuel, pero teniendo en

+ Mensaje para ChatGPT

Instantánea



Sí. Arranquemos con **nuestro Bloque 1 en MySQL**, equivalente al primer bloque de Samuel, pero teniendo en cuenta que nosotros ya creamos `brechas_seguridad` y estamos trabajando en AWS RDS. En Samuel este primer bloque crea usuario/esquema y configura privilegios.

Texto pegado.txt
TXT

Como en MySQL una `DATABASE` cumple aquí el papel que necesitamos para separar nuestras tablas, y ya comprobamos que `SELECT DATABASE()` devuelve `brechas_seguridad`, no necesitamos volver a crearla.

Pega esto en tu editor de DBeaver:

```
</> SQL
-- =====
-- Examen 02 - Tópicos Avanzados de Base de Datos
-- Proyecto Avanzado N1
-- MySQL 8.4.9 - AWS RDS
-- =====

-- =====
-- BLOQUE 1. BASE DE DATOS
-- =====

USE brechas_seguridad;

SELECT DATABASE();
```

Ejecútalo

Esta vez hazlo como **script completo**, no como una sola sentencia.

En DBeaver puedes seleccionar las dos instrucciones:

```
</> SQL
USE brechas_seguridad;

SELECT DATABASE();
```

y ejecutar el script.

El resultado final debe mostrar:

```
</> Texto sin formato
DATABASE()
-----
brechas_seguridad
```

Eso ya lo habíamos conseguido en tu captura, así que **Bloque 1 está listo**. ✓

Ahora viene el importante: **Bloque 2**, donde vamos a convertir las 5 tablas normalizadas de Samuel de PostgreSQL a MySQL, conservando PK, FK, `UNIQUE` y `CHECK`. Samuel empieza esas tablas en `organizacion` y termina en `exposicion_usuario_brecha`.

Texto pegado.txt
TXT

Ese es el que debemos montar ahora.

📄 ↶ ↷ 📄 🔍 ...

que tal si me lo as todo todo y vamos ejecutando lo que tu me digas

Sí, mejor así. Yo te voy armando **todo el 01_implementation.sql** de MySQL, manteniendo la estructura y lógica del de Samuel y cambiando únicamente lo necesario por diferencias PostgreSQL → MySQL.

Pero **no lo ejecutamos todo** todavía, porque en medio necesariamente hay que importar los CSV al staging. Samuel también tiene esa pausa.

Texto pegado.txt
TXT

Vamos por bloques y yo te digo exactamente qué seleccionar y ejecutar.

Paso 1 — Ejecuta Bloques 1, 2 y 3

Borra lo que tengas en ese editor y pega **todo esto**:

```
</> SQL
-- =====
-- Examen 02 - Tópicos Avanzados de Base de Datos
-- Proyecto Avanzado N1
-- MySQL 8.4.9 - AWS RDS
-- =====
```

+ Mensaje para ChatGPT

Instantánea ~ 🔍 ↶

Pero no lo ejecutamos todo todavía, porque en medio necesariamente hay que importar los CSV al staging. Samuel también tiene esa pausa.

Texto pegado.txt
TXT

Vamos por bloques y yo te digo exactamente qué seleccionar y ejecutar.

Paso 1 — Ejecuta Bloques 1, 2 y 3

Borra lo que tengas en ese editor y pega todo esto:

```
</> SQL

-- =====
-- Examen 02 - Topicos Avanzados de Base de Datos
-- Proyecto Avanzado N1
-- MySQL 8.4.9 - AWS RDS
-- =====

-- =====
-- BLOQUE 1. BASE DE DATOS
-- =====

USE brechas_seguridad;

-- =====
-- BLOQUE 2. TABLAS NORMALIZADAS
-- =====

CREATE TABLE organizacion (
  id_organizacion INT NOT NULL,
  nombre_organizacion VARCHAR(200) NOT NULL,
  sector VARCHAR(100) NOT NULL,
  pais VARCHAR(100) NOT NULL,

  CONSTRAINT pk_organizacion
    PRIMARY KEY (id_organizacion),

  CONSTRAINT uq_organizacion_nombre
    UNIQUE (nombre_organizacion)
);

CREATE TABLE brecha (
  codigo_brecha VARCHAR(20) NOT NULL,
  id_organizacion INT NOT NULL,
  fecha_deteccion DATE NOT NULL,
  vector_ataque VARCHAR(100) NOT NULL,
  registros_comprometidos INT NOT NULL,
  costo_estimado_usd DECIMAL(15,2) NOT NULL,
  severidad_incidente VARCHAR(20) NOT NULL,

  CONSTRAINT pk_brecha
    PRIMARY KEY (codigo_brecha),

  CONSTRAINT fk_brecha_organizacion
    FOREIGN KEY (id_organizacion)
      REFERENCES organizacion (id_organizacion),

  CONSTRAINT chk_brecha_registros
    CHECK (registros_comprometidos >= 0),

  CONSTRAINT chk_brecha_costo
    CHECK (costo_estimado_usd >= 0),

  CONSTRAINT chk_brecha_severidad
    CHECK (
      severidad_incidente IN
      ('Baja', 'Media', 'Alta', 'Critica')
    )
);

CREATE TABLE usuario_afectado (
  codigo_usuario VARCHAR(20) NOT NULL,
  pais_residencia VARCHAR(100) NOT NULL,
  rango_edad VARCHAR(50) NOT NULL,

  CONSTRAINT pk_usuario_afectado
    PRIMARY KEY (codigo_usuario)
);

CREATE TABLE tipo_dato_expuesto (
  id_tipo_dato INT NOT NULL,
  nombre_tipo_dato VARCHAR(150) NOT NULL,
  categoria_sensibilidad VARCHAR(20) NOT NULL,
```

+ Mensaje para ChatGPT

Instantánea ~





Casi perfecto. La captura muestra:

- Lote 4 → `staging_brechas` [Existing] ✓
- Lote 3 → `staging_brechas` [Existing] ✓
- Lote 2 → `staging_brechas` [Existing] ✓
- Lote 1 → `sabana_brechas_seguridad_lote` [Create] ✗

Solo falta cambiar el lote 1 exactamente igual que hiciste con los otros tres, hasta que aparezca:

```
brechas_seguridad.staging_brechas [Existing]
```

Cuando los 4 digan `[Existing]`, mándame captura y seguimos con **Proceed**. No lo pulses todavía.

📄 ⬆️ ↻ 🗑️ ...



Perfecto. Ahora sí están los 4 correctamente configurados. ✓

Los cuatro lotes apuntan a:

```
brechas_seguridad.staging_brechas [Existing]
```

Eso significa que todos los CSV se van a cargar en la misma tabla `staging_brechas`, que es exactamente lo que necesitamos.

Siguiente paso

Ahora sí dale a **Proceed**.

Deja que termine completamente la importación. Cuando DBeaver te muestre el resultado final —sea exitoso o aparezca algún error— mándame captura y no hagas nada más. Ahí verificamos cuántas filas quedaron cargadas antes de continuar con el siguiente bloque del examen.

📄 ⬆️ ↻ 🗑️ ...

carga desde `staging_brechas` hacia las tablas normalizadas.

📄 📤 🔄 🗑️ ...



```
python3 load_data.py --table staging_brechas --table organization --table brecha --table usuario_afectado --table tipo_dato_expuesto --table exposicion_usuario_brecha
```

Perfecto. Ahora sí quedó bien la carga completa. ✅

La captura confirma:

- `total` → 40.000
- `con_organizacion` → 40.000
- `con_sector` → 40.000
- `con_pais` → 40.000
- `con_brecha` → 40.000
- `con_fecha_ocurrencia` → 40.000

Así que los 4 lotes quedaron correctamente consolidados en `staging_brechas`, y solucionamos el problema de las columnas que estaban quedando en `NULL`.

Siguiente paso

Ahora viene lo importante: pasar los datos de `staging_brechas` a las tablas normalizadas:

`organizacion`
`brecha`
`usuario_afectado`
`tipo_dato_expuesto`
`exposicion_usuario_brecha`

Pero no ejecutemos nada todavía a ciegas. Como ya tenemos el staging perfecto, el siguiente bloque debe respetar exactamente las PK, FK y restricciones que creamos.

Vamos a hacerlo tabla por tabla y verificando cada carga, para que si alguna restricción falla sepamos exactamente dónde.

Empezamos con `organizacion`.

📄 📤 🔄 🗑️ ...

vamos

Vamos. Como el `staging_brechas` ya quedó perfecto, ahora vamos paso por paso y verificamos cada tabla antes de seguir.

1. Primero revisemos cuántas organizaciones distintas tenemos

Ejecuta solo esto:

```
</> SQL
SELECT COUNT(DISTINCT nombre_organizacion) AS organizaciones_distintas
FROM staging_brechas;
```

Todavía no insertes nada.

Mándame el número que te salga y con eso hacemos inmediatamente el `INSERT` correcto en `organizacion`, teniendo en cuenta que `id_organizacion` es la PK.

📄 📤 🔄 🗑️ ...

salio 60

Perfecto: 60 organizaciones distintas. Ahora carguemos `organizacion`.

Ejecuta este bloque:

```
</> SQL
INSERT INTO organizacion (
  id_organizacion,
  nombre_organizacion,
  sector,
  pais
)
SELECT
  ROW_NUMBER() OVER (ORDER BY nombre_organizacion) AS id_organizacion,
  nombre_organizacion,
  MAX(sector_organizacion) AS sector,
  MAX(pais_organizacion) AS pais
FROM staging_brechas
GROUP BY nombre_organizacion;
```

+ Mensaje para ChatGPT

Instantánea

🗑️ 📤

Ejecuta este bloque:

```
</> SQL
INSERT INTO organizacion (
  id_organizacion,
  nombre_organizacion,
  sector,
  pais
)
SELECT
  ROW_NUMBER() OVER (ORDER BY nombre_organizacion) AS id_organizacion,
  nombre_organizacion,
  MAX(sector_organizacion) AS sector,
  MAX(pais_organizacion) AS pais
FROM staging_brechas
GROUP BY nombre_organizacion;
```

Si ejecuta correctamente, verifica:

```
</> SQL
SELECT COUNT(*) AS total_organizaciones
FROM organizacion;
```

👉 Debe darte 60.

Si da 60, me dices y pasamos a la tabla `brecha`.

📄 ↗ ↻ 🔍 ...

si salio

Perfecto, entonces `organizacion` quedó cargada con 60 registros. ✅

Ahora seguimos con `brecha`. Primero vamos a verificar cuántas brechas únicas tenemos antes de insertar nada.

Ejecuta:

```
</> SQL
SELECT COUNT(DISTINCT codigo_brecha) AS brechas_distintas
FROM staging_brechas;
```

Dime cuánto te sale y hacemos el `INSERT` de `brecha`.

📄 ↗ ↻ 🔍 ...

800

📄 ✎

Perfecto. 800 brechas distintas. ✅

Antes de insertar, vamos a hacerlo igual que con `organizacion`: cargar las 800 brechas relacionándolas con su organización.

Ejecuta:

```
</> SQL
INSERT INTO brecha (
  codigo_brecha,
  id_organizacion,
  fecha_ocurrencia,
  fecha_deteccion,
  vector_ataque,
  severidad_incidente,
  registros_afectados_total,
  costo_estimado_total
)
SELECT
  s.codigo_brecha,
  o.id_organizacion,
  MAX(s.fecha_ocurrencia),
  MAX(s.fecha_deteccion),
  MAX(s.vector_ataque),
  MAX(s.severidad_incidente),
  MAX(s.registros_afectados_total),
  MAX(s.costo_estimado_total)
FROM staging_brechas s
JOIN organizacion o
  ON o.nombre_organizacion = s.nombre_organizacion
GROUP BY
  s.codigo_brecha,
```

+ Mensaje para ChatGPT

Instantánea ▾



bie pero recuerda que mi amigo samuel y tiene los codigos o el script pero en postgres

Sí, lo tengo presente. Samuel tiene el script/códigos resueltos en PostgreSQL, mientras que tú estás haciendo el examen en MySQL sobre AWS RDS.

Eso nos sirve bastante como referencia. La idea para lo que falta será:

1. Tomar la lógica del código de Samuel.
2. Entender qué está haciendo, no copiarlo ciegamente.
3. Convertir lo necesario de PostgreSQL → MySQL.
4. Adaptarlo a tus tablas reales (organizacion, brecha, usuario_afectado, tipo_dato_expuesto, exposicion_usuario_brecha, etc.).
5. Ir ejecutando punto por punto y verificar los resultados en tu RDS.

Así evitamos errores por diferencias entre PostgreSQL y MySQL, especialmente en índices, EXPLAIN, procedimientos/funciones y sintaxis de performance tuning.

Y como ya dejamos tus datos correctamente cargados, estamos en muy buena posición para empezar esa parte. Pásame el primer código/punto de Samuel y lo adaptamos a MySQL paso a paso.

📄 ⬆️ ↻ 🔍 ...



tiene estos 3 cual te paso? creo que el tercero el de implementacion ya lo hicimos no?

Sí, exactamente. El tercero, implementacion_modelo_postgresql.sql, prácticamente ya lo hicimos nosotros en MySQL: crear/usar el modelo, cargar los 4 lotes en staging_brechas y distribuir los datos entre las tablas normalizadas.

Ahora me interesan principalmente estos dos:

1. etapa3_consulta_brechas_criticas.sql ← pásame este primero. Lo revisamos y convertimos de PostgreSQL a MySQL.
2. Después etapa4_historial_exposicion_usuario.sql.

Vamos en ese orden para no mezclar etapas. Pásame etapa3_consulta_brechas_criticas.sql.

📄 ⬆️ ↻ 🔍 ...

```
-- Examen 02 - Topicos Avanzados de Base de Datos
-- Etapa 3 - Brechas criticas del ultimo ano
-- Samuel Parra Cadavid
-- ID SIGAA: 000551191
-- PostgreSQL 18.x

-- Consulta base
select
&#x20; o.nombre_organizacion,
&#x20; b.codigo_brecha,
&#x20; b.fecha_deteccion,
&#x20; b.vector_ataque,
&#x20; count(distinct e.codigo_usuario) as usuarios_afectados
from brechas_seguridad.brecha b
join brechas_seguridad.organizacion o
&#x20; on o.id_organizacion = b.id_organizacion
join brechas_seguridad.exposicion_usuario_brecha e
&#x20; on e.codigo_brecha = b.codigo_brecha
where b.fecha_deteccion >= (
...
Mostrar más ▾
```

📄 ✎

Sí. Este es claramente el punto/Etapa 3 de Performance Tuning. La lógica de Samuel nos sirve casi completa, pero hay dos cambios importantes para MySQL:

- PostgreSQL: INTERVAL '1 year' → MySQL: INTERVAL 1 YEAR
- PostgreSQL: EXPLAIN (ANALYZE, BUFFERS) → MySQL: EXPLAIN ANALYZE

Vamos paso por paso, para que además tengas las evidencias del antes y después.

Paso 1: consulta base en MySQL

Primero NO creemos ningún índice nuevo. Ejecuta solamente esto:

```
</> SQL
SELECT
  o.nombre_organizacion,
  b.codigo_brecha,
  b.fecha_deteccion,
  ↓
+ Mensaje para ChatGPT Instantánea 🔊 ↗
```

Sí. Este es claramente el **punto/Etapa 3 de Performance Tuning**. La lógica de Samuel nos sirve casi completa, pero hay dos cambios importantes para MySQL:

- PostgreSQL: `INTERVAL '1 year'` → MySQL: `INTERVAL 1 YEAR`
- PostgreSQL: `EXPLAIN (ANALYZE, BUFFERS)` → MySQL: `EXPLAIN ANALYZE`

Vamos **paso por paso**, para que además tengas las evidencias del antes y después.

Paso 1: consulta base en MySQL

Primero **NO** creemos ningún índice nuevo. Ejecuta solamente esto:

```
</> SQL
SELECT
  o.nombre_organizacion,
  b.codigo_brecha,
  b.fecha_deteccion,
  b.vector_ataque,
  COUNT(DISTINCT e.codigo_usuario) AS usuarios_afectados
FROM brechas_seguridad.brecha b
JOIN brechas_seguridad.organizacion o
  ON o.id_organizacion = b.id_organizacion
JOIN brechas_seguridad.exposicion_usuario_brecha e
  ON e.codigo_brecha = b.codigo_brecha
WHERE b.fecha_deteccion >= (
  SELECT MAX(fecha_deteccion) - INTERVAL 1 YEAR
  FROM brechas_seguridad.brecha
)
AND b.severidad_incidente IN ('Alta', 'Critica')
AND EXISTS (
  SELECT 1
  FROM brechas_seguridad.exposicion_usuario_brecha e_critica
  JOIN brechas_seguridad.tipo_dato_expuesto t_critico
    ON t_critico.id_tipo_dato = e_critica.id_tipo_dato
  WHERE e_critica.codigo_brecha = b.codigo_brecha
    AND t_critico.categoria_sensibilidad = 'Critica'
)
GROUP BY
  o.nombre_organizacion,
  b.codigo_brecha,
  b.fecha_deteccion,
  b.vector_ataque
ORDER BY usuarios_afectados DESC;
```

Esta consulta busca, igual que la de Samuel:

brechas detectadas durante el último año del dataset, de severidad **Alta** o **Critica**, que además hayan expuesto al menos un tipo de dato de sensibilidad **Critica**; luego cuenta los usuarios distintos afectados por cada brecha.

Importante: el "último año" **no se calcula desde hoy**, sino desde:

```
</> SQL
MAX(fecha_deteccion)
```

Eso está bien porque así lo plantea el script de Samuel.

Paso 2

Cuando esa consulta funcione, **no crees todavía `idx_brecha_fecha_severidad`**.

Lo siguiente será sacar el plan inicial con:

```
</> SQL
EXPLAIN ANALYZE
SELECT ...
```

Este resultado será nuestra evidencia **ANTES de optimizar**. Después creamos el primer índice, volvemos a medir; luego el segundo índice y medimos nuevamente.

Así podremos comparar realmente:

Sin índices → Índice 1 → Índices 1 + 2.

Primero ejecuta la consulta de arriba y muéstrame qué resultado te da.

dime que voy ejecutandfo



Vamos uno por uno para ir guardando las evidencias.

Paso 1 — Consulta base

Ejecuta solo esto primero:

```
</> SQL
SELECT
  o.nombre_organizacion,
  b.codigo_brecha,
  b.fecha_deteccion,
  b.vector_ataque,
  COUNT(DISTINCT e.codigo_usuario) AS usuarios_afectados
FROM brechas_seguridad.brecha b
JOIN brechas_seguridad.organizacion o
  ON o.id_organizacion = b.id_organizacion
JOIN brechas_seguridad.exposicion_usuario_brecha e
  ON e.codigo_brecha = b.codigo_brecha
WHERE b.fecha_deteccion >= (
  SELECT MAX(fecha_deteccion) - INTERVAL 1 YEAR
  FROM brechas_seguridad.brecha
)
AND b.severidad_incidente IN ('Alta', 'Critica')
AND EXISTS (
  SELECT 1
  FROM brechas_seguridad.exposicion_usuario_brecha e_critica
  JOIN brechas_seguridad.tipo_dato_expuesto t_critico
    ON t_critico.id_tipo_dato = e_critica.id_tipo_dato
  WHERE e_critica.codigo_brecha = b.codigo_brecha
    AND t_critico.categoria_sensibilidad = 'Critica'
)
GROUP BY
  o.nombre_organizacion,
  b.codigo_brecha,
  b.fecha_deteccion,
  b.vector_ataque
ORDER BY usuarios_afectados DESC;
```

Todavía NO ejecutes ningún **CREATE INDEX** .

Cuando termine, mándame captura del resultado y seguimos con el Paso 2: **EXPLAIN ANALYZE** inicial.



```
Reed Group BR-00408 2026-09-22 Insider Malicioso 58
Gallegos-Wilkerson BR-00133 2026-03-14 Ataque de Fuerza Bruta
55
Combs, Kent and Higgins BR-00432 2026-06-28 Phishing 54
Walton LLC BR-00682 2026-09-22 Configuracion Erronea de Servidor
52
Ward Ltd BR-00715 2026-04-19 Insider Malicioso 52
Combs, Kent and Higgins BR-00036 2026-02-03 Credenciales
Filtradas de Terceros 51
Combs, Kent and Higgins BR-00247 2026-02-09 Phishing 51
Cruz PLC BR-00412 2026-09-22 Configuracion Erronea de Servidor
50
Cruz PLC BR-00417 2026-04-19 Phishing 50
Hughes LLC BR-00478 2026-07-14 Phishing 49
Lynn-Garcia BR-00286 2026-09-22 Phishing 49
Baker-Stewart BR-00005 2025-10-10 Insider Malicioso 48
Freeman-Martin BR-00541 2026-07-07 Configuracion Erronea de
Servidor 48
Murphy-Mullins BR-00332 2026-09-22 Insider Malicioso 48
...
Mostrar más ▾
```

Perfecto. Eso confirma que la **consulta base funciona correctamente en MySQL** y está devolviendo las brechas críticas/altas del último año con los usuarios afectados.

Ahora **no crees índices todavía**. Necesitamos obtener el rendimiento inicial para después demostrar la mejora.

Paso 2 — Plan de ejecución inicial

Ejecuta exactamente esto:

```
</> SQL
EXPLAIN ANALYZE
SELECT
  o.nombre_organizacion,
```



+ Mensaje para ChatGPT

Instantánea ▾



Perfecto. Esta Etapa 4 es más corta. La lógica de Samuel se mantiene, pero hay una diferencia importante: **MySQL no soporta `INCLUDE (...)` en `CREATE INDEX`** como PostgreSQL. Para conseguir un índice cubriendo equivalente, ponemos esas columnas al final del índice compuesto.

Te dejo primero el archivo completo equivalente para MySQL y luego lo ejecutamos por partes:

```
</> SQL

-- Examen 02 - Topicos Avanzados de Base de Datos
-- Etapa 4 - Historial de exposicion de un usuario
-- MySQL

-- Consulta funcional
SELECT
    u.codigo_usuario,
    b.codigo_brecha,
    t.nombre_tipo_dato,
    t.categoria_sensibilidad,
    o.nombre_organizacion,
    e.fecha_notificacion_usuario
FROM brechas_seguridad.usuario_afectado u
JOIN brechas_seguridad.exposicion_usuario_brecha e
    ON e.codigo_usuario = u.codigo_usuario
JOIN brechas_seguridad.brecha b
    ON b.codigo_brecha = e.codigo_brecha
JOIN brechas_seguridad.tipo_dato_expuesto t
    ON t.id_tipo_dato = e.id_tipo_dato
JOIN brechas_seguridad.organizacion o
    ON o.id_organizacion = b.id_organizacion
WHERE u.codigo_usuario = 'US-002823'
ORDER BY e.fecha_notificacion_usuario;

-- Plan de ejecucion inicial
EXPLAIN ANALYZE
SELECT
    u.codigo_usuario,
    b.codigo_brecha,
    t.nombre_tipo_dato,
    t.categoria_sensibilidad,
    o.nombre_organizacion,
    e.fecha_notificacion_usuario
FROM brechas_seguridad.usuario_afectado u
JOIN brechas_seguridad.exposicion_usuario_brecha e
    ON e.codigo_usuario = u.codigo_usuario
JOIN brechas_seguridad.brecha b
    ON b.codigo_brecha = e.codigo_brecha
JOIN brechas_seguridad.tipo_dato_expuesto t
    ON t.id_tipo_dato = e.id_tipo_dato
JOIN brechas_seguridad.organizacion o
    ON o.id_organizacion = b.id_organizacion
WHERE u.codigo_usuario = 'US-002823'
ORDER BY e.fecha_notificacion_usuario;

-- Indice de optimizacion
CREATE INDEX idx_exposicion_usuario_fecha
ON brechas_seguridad.exposicion_usuario_brecha (
    codigo_usuario,
    fecha_notificacion_usuario,
    codigo_brecha,
    id_tipo_dato
);

-- Plan de ejecucion optimizado
EXPLAIN ANALYZE
SELECT
    u.codigo_usuario,
    b.codigo_brecha,
    t.nombre_tipo_dato,
    t.categoria_sensibilidad,
    o.nombre_organizacion,
    e.fecha_notificacion_usuario
FROM brechas_seguridad.usuario_afectado u
JOIN brechas_seguridad.exposicion_usuario_brecha e
    ON e.codigo_usuario = u.codigo_usuario
JOIN brechas_seguridad.brecha b
    ON b.codigo_brecha = e.codigo_brecha
JOIN brechas_seguridad.tipo_dato_expuesto t
    ON t.id_tipo_dato = e.id_tipo_dato
JOIN brechas_seguridad.organizacion o
    ON o.id_organizacion = b.id_organizacion
WHERE u.codigo_usuario = 'US-002823'
ORDER BY e.fecha_notificacion_usuario;
```

Vamos ejecutando

Paso 1: consulta equivalente de postgres. Consulta funcional dada (PGSQL) hasta:

+ Mensaje para ChatGPT

Instantánea

Selecciona **todo este bloque completo** y ejecútalo:

```
</> SQL
SELECT
  u.codigo_usuario,
  b.codigo_brecha,
  t.nombre_tipo_dato,
  t.categoria_sensibilidad,
  o.nombre_organizacion,
  e.fecha_notificacion_usuario
FROM brechas_seguridad.usuario_afectado u
JOIN brechas_seguridad.exposicion_usuario_brecha e
  ON e.codigo_usuario = u.codigo_usuario
JOIN brechas_seguridad.brecha b
  ON b.codigo_brecha = e.codigo_brecha
JOIN brechas_seguridad.tipo_dato_expuesto t
  ON t.id_tipo_dato = e.id_tipo_dato
JOIN brechas_seguridad.organizacion o
  ON o.id_organizacion = b.id_organizacion
WHERE u.codigo_usuario = 'US-002823'
ORDER BY e.fecha_notificacion_usuario;
```

Fíjate que en tu captura aparece arriba resaltado solo `ORDER BY...`, por eso ocurrió el error `1064`.

Ejecuta el bloque entero y dime **qué filas te devuelve**. Todavía no hagamos el índice.

📄 🔄 ⚙️ ...

```
US-002823  BR-00088  Numero de Identificacion  Alta  Lynn-Garcia
2022-03-03
US-002823  BR-00165  Historial Medico Critica  Lynn-Garcia  2022-
07-18
US-002823  BR-00570  Numero de Identificacion  Alta  Lynn-Garcia
2022-08-18
US-002823  BR-00571  Contraseña Alta  Lynn-Garcia  2022-10-04
US-002823  BR-00181  Numero de Identificacion  Alta  Lynn-Garcia
2023-08-01
US-002823  BR-00507  Correo Electronico Media  Lynn-Garcia  2023-
09-24
US-002823  BR-00507  Contraseña Alta  Lynn-Garcia  2023-10-11
US-002823  BR-00198  Historial de Transacciones  Alta  Lynn-Garcia
2024-01-11
US-002823  BR-00501  Fecha de Nacimiento Baja  Lynn-Garcia  2024-
06-08
US-002823  BR-00612  Contraseña Alta  Lynn-Garcia  2024-06-20
US-002823  BR-00348  Numero Telefonico Baja  Lynn-Garcia  2024-
07-20
...
Mostrar más ▾
```

📄 ✎

Perfecto. ✅ La consulta funcional de la Etapa 4 está correcta: devolvió 16 exposiciones del usuario `US-002823`, ordenadas por `fecha_notificacion_usuario`.

Ahora necesitamos medir el rendimiento **antes de crear el índice**.

Paso 2 — Plan inicial

Ejecuta este bloque completo:

```
</> SQL
EXPLAIN ANALYZE
SELECT
  u.codigo_usuario,
  b.codigo_brecha,
  t.nombre_tipo_dato,
  t.categoria_sensibilidad,
  o.nombre_organizacion,
  e.fecha_notificacion_usuario
FROM brechas_seguridad.usuario_afectado u
JOIN brechas_seguridad.exposicion_usuario_brecha e
  ON e.codigo_usuario = u.codigo_usuario
JOIN brechas_seguridad.brecha b
  ON b.codigo_brecha = e.codigo_brecha
JOIN brechas_seguridad.tipo_dato_expuesto t
  ON t.id_tipo_dato = e.id_tipo_dato
JOIN brechas_seguridad.organizacion o
  ON o.id_organizacion = b.id_organizacion
WHERE u.codigo_usuario = 'US-002823'
ORDER BY e.fecha_notificacion_usuario;
```

⚠️ Todavía no crees `idx_exposicion_usuario_fecha`.

Pásame lo que salga en el `EXPLAIN ANALYZE`. Nos inténta especialmente el `actual time` final y cómo está accediendo a `exposicion_usuario_brecha`.

+ Mensaje para ChatGPT

Instantánea ▾ 🔍 ⬆

Perfecto. Ya tenemos la medición **antes de optimizar**:

- Tiempo final: ≈ 0.275 ms
- Filas: 16
- MySQL ya usa el índice `fk_exposicion_usuario` para encontrar el usuario.
- Pero después necesita hacer un **Sort** por `fecha_notificacion_usuario`.

Eso último es justamente lo que intentaremos mejorar con el índice de Samuel.

Paso 3 — Crear el índice de optimización

En PostgreSQL Samuel usó `INCLUDE`. En MySQL lo adaptamos poniendo esas columnas dentro del índice:

```
</> SQL
CREATE INDEX idx_exposicion_usuario_fecha
ON brechas_seguridad.exposicion_usuario_brecha (
  codigo_usuario,
  fecha_notificacion_usuario,
  codigo_brecha,
  id_tipo_datos
);
```

Ejecuta solo eso.

Cuando salga correctamente, dime `listo`. Después repetimos el `EXPLAIN ANALYZE` y comprobamos si desaparece este:

```
</> Texto sin formato
Sort: e.fecha_notificacion_usuario
```

y si MySQL empieza a utilizar `idx_exposicion_usuario_fecha`.

📄 ↕ 🔄 ⚙️ ...

```
-> Nested loop inner join (cost=19.7 rows=16) (actual time=0.0644..0.163
rows=16 loops=1)
  &#x20; -> Nested loop inner join (cost=14.1 rows=16) (actual
time=0.0574..0.15 rows=16 loops=1)
  &#x20; -> Nested loop inner join (cost=8.52 rows=16) (actual
time=0.0399..0.0662 rows=16 loops=1)
  &#x20; -> Covering index lookup on e using
idx_exposicion_usuario_fecha (codigo_usuario='US-002823')
(cost=2.92 rows=16) (actual time=0.0309..0.0366 rows=16 loops=1)
  &#x20; -> Single-row index lookup on t using PRIMARY
(id_tipo_datos=e.id_tipo_datos) (cost=0.256 rows=1) (actual
time=0.00155..0.00158 rows=1 loops=16)
  &#x20; -> Single-row index lookup on b using PRIMARY
(codigo_brecha=e.codigo_brecha) (cost=0.256 rows=1) (actual
time=0.005..0.00504 rows=1 loops=16)
  &#x20; -> Single-row index lookup on o using PRIMARY
(id_organizacion=b.id_organizacion) (cost=0.256 rows=1) (actual
time=529e-6..566e-6 rows=1 loops=16)
```

Excelente. Etapa 4 quedó funcionando y la optimización aquí sí fue clarísima. ✅

Comparación

Antes del índice:

```
</> Texto sin formato
actual time = 0.177..0.275 ms
```

Además MySQL tenía que hacer:

```
</> Texto sin formato
Sort: e.fecha_notificacion_usuario
```

Después del índice:

```
</> Texto sin formato
actual time = 0.0644..0.163 ms
```

Y ahora aparece:

```
</> Texto sin formato
Covering index lookup on e
using idx_exposicion_usuario_fecha
(codigo_usuario='US-002823')
```

+ Mensaje para ChatGPT

Instantánea

